

A Contact-Safe Reinforcement Learning Framework for Contact-Rich Robot Manipulation

Xiang Zhu, Shucheng Kang and Jianyu Chen*

Abstract—Reinforcement learning shows great potential to solve complex contact-rich robot manipulation tasks. However, the safety of using RL in the real world is a crucial problem, since unexpected dangerous collisions might happen when the RL policy is imperfect during training or in unseen scenarios. In this paper, we propose a contact-safe reinforcement learning framework for contact-rich robot manipulation, which maintains safety in both the task space and joint space. When the RL policy causes unexpected collisions between the robot arm and the environment, our framework is able to immediately detect the collision and ensure the contact force to be small. Furthermore, the end-effector is enforced to perform contact-rich tasks compliantly, while keeping robust to external disturbances. We train the RL policy in simulation and transfer it to the real robot. Real world experiments on robot wiping tasks show that our method is able to keep the contact force small both in task space and joint space even when the policy is under unseen scenario with unexpected collision, while rejecting the disturbances on the main task.

I. INTRODUCTION

As the demand of automation in both industrial and daily applications increase rapidly, robots are required to accomplish more and more complex manipulation tasks. These tasks generally involve rich contacts with various unknown objects in unstructured environments, which makes the modeling and control of the robots quite challenging. Reinforcement Learning (RL), on the other hand, provides a model-free paradigm to solve complex tasks in unknown environments by self exploration and reinforcement. Several recent works have show great potential in applying RL to learn contact-rich robot manipulation policies, such as grasping [1] [2], insertion [3] [4] and assembling [5] [6].

Although quite promising, a major reason that prevents the wide use of RL in real world applications is its lack of safety. During the exploration procedure of RL, a common approach is to take some random actions [7] or try to visit unexperienced states [8], which, however, may lead to some unexpected dangers. Even if the training process of RL converges, the resulted policies are still prone to generate unexpected dangerous behaviors when met with unseen scenarios, and easily get interfered by external disturbances. For example, as shown in Fig. 1 (the two upper subfigures), an RL policy has been trained to wipe the ink on the whiteboard on the left. However, when there is an unseen object on the whiteboard, the policy is disturbed and the robot moves to the right. These unexpected behaviors of RL may result in

Xiang Zhu, Shucheng Kang and Jianyu Chen are with the institute for interdisciplinary information sciences, Tsinghua University, Beijing, China. Jianyu Chen is also with the Shanghai Qizhi Insitute, Shanghai, China. (*corresponding author, Email: jianyuchen@tsinghua.edu.cn)



Fig. 1. Safety under unseen scenario: Our trained RL agent can finish the wiping task properly (shown in the top left figure). However, when an unseen object appears (shown in the top right figure), the RL policy fails and unexpectedly command the robot to move right. Such behavior might cause some unexpected contact. In this experiment, the robot collides with a plastic water bottle we grab. The green border pictures show that our method generate compliant contact behavior with small contact force. However the baseline, as shown in the red border figures, generate nearly 3 times the contact force of our method. As we can see, the baseline method significantly deforms the water bottle while our method does not deform it at all. The bottom left two figures show the contact force using a thrush meter and the bottom right two figures show the deformation of the water bottle.

unexpected collisions accompanying with large contact force between the robots and the environments (or even people), causing huge damages. Therefore, in this work we focus on such contact-safety when applying RL to contact-rich robot manipulation.

According to where the dangerous contact might happen, the contact-safety can be categorized into task space safety and joint space safety. The task space safety requires the contact to be compliant between the robot end-effector and the targeted objects on which it is performing the tasks. Such contact compliance can be achieved with impedance control [9], which aims to control the dynamic relationship between the robot and the contact objects as a second order

system consisting of a virtual mass, spring and damper. Variable impedance control (VIC) [10] allows adjusting the stiffness according to demand, showing stronger potential for ensuring contact safety. Combining with RL, [11] shows that using VIC in end-effector space as action space results in better sample efficiency, low energy consumption and more compliant end-effector contacts. Other related works include combining VIC and RL to perform high-precision robotic assembly tasks [6], and combining model-based RL and VIC to enable safe collaboration between the robot's end-effector and human [12]. However, all these works care about the task space safety problem solely, while ignoring the crucial joint space safety problem in RL.

Joint space safety focuses on the potential dangerous contacts happening at the robot arm, which can be common in human-robot interaction tasks or unstructured environments. Existing related works target on totally avoiding contact when using RL. Q. Yang et al. [13] ensure collision avoidance safety by only learning policies in the null space of higher prioritized constraints. S. Shaw et al. [14] combine VIC and Riemannian motion policies (RMPs) which use geometric information of the arm and the surrounding environment to generate collision-free joint actions. [15] [16] focus on the Human-Robot Collaboration scenario which also aims to find collision-free actions. However, avoiding all collisions requires precise geometric information about the object, which is expensive and difficult to obtain. Furthermore, in cluttered and unknown environments, unexpected contacts might be inevitable. In some certain scenarios, we even need to create contact to better accomplish tasks [17]. In these cases, the joint space safety should be achieved not by avoiding collisions but by limiting the magnitude of the contact force. Nevertheless, to the best of our knowledge, no prior works have considered such joint space contact safety when using reinforcement learning.

In this paper, we propose a contact-safe reinforcement learning framework for contact-rich robot manipulation tasks, where both the task space safety and joint space safety are handled when using RL. Our framework is compatible with any RL algorithms, while in this work we use an image-based RL algorithm which learns a convolutional neural network (CNN) policy taking images as inputs and performing surface wiping tasks. The policy's output belongs to the Cartesian variable impedance control action space which helps generate compliant contact at the end-effector. A momentum observer is developed to estimate the contact point and force applied on the robot arm, purely from joint measurements. When collision is detected, a contact-aware controller will be activated, which performs a contact-aware null space projection that significantly reduces the contact force (as shown in Fig. 1). Meanwhile, the disturbance compensation impedance term helps reject the disturbance to the main task. The RL policy is trained in simulation and transferred to the real robot. Real world experiment results show that compared to the baseline, our method produces significantly more compliant contact force during collision and less error during disturbances.

II. PRELIMINARY

A. Reinforcement Learning

The goal of Reinforcement Learning is to find a reward maximizing policy π that conditioned only on the current time step [7]. This problem can be modeled as a Markov decision process (MDP) which is represented by a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, \mu)$, where $\mathcal{S}$, $\mathcal{A}$ are the state and action spaces respectively, $\mathcal{P}$ represents the transition model that maps the state and action into a probability distribution of the next state $P(s'|s, a)$, $\mathcal{R}$ represents the reward function $r(s, a)$, γ is the discount factor for the reward function and μ represents the initial state distribution. We also define the discounted future return $R(\tau) = \sum_{i=t}^T \gamma^{i-t} r(s_i, a_i)$, where $\tau = (s_t, a_t \dots s_T, a_T)$. Assume our policy π is parameterized by θ , then the RL algorithm optimizes θ by maximizing the expected future return:

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{\tau \sim \pi(\theta)} R(\tau) \quad (1)$$

B. Operational Space Formulation and Variable Impedance Control

Operational space formulation [18] is widely used in torque controlled robots. The equation of motion of an open chain robot manipulator can be written as:

$$M(q)\ddot{q} + C(q, \dot{q}) + g(q) = \tau_m - \tau_{ext} \quad (2)$$

where $q \in \mathbb{R}^n$ represents the generalized coordinates, $M(q) \in \mathbb{R}^{n \times n}$ is the positive-definite and symmetric mass matrix, $C(q, \dot{q}) \in \mathbb{R}^n$ is the centrifugal and Coriolis term, $g(q) \in \mathbb{R}^n$ is the gravity vector, $\tau_m \in \mathbb{R}^n$ and $\tau_{ext} \in \mathbb{R}^n$ on the right-hand side is the applied torque and the external joint torque.

Considering an manipulator with task coordinates t , we can write the mapping from generalized coordinates to task coordinates as $\dot{x} = J_t(q)\dot{q}$, where J_t is the task Jacobian and is assumed be full row-rank. We can write the task mass matrix as $\Lambda_t(q) = (J_t(q)M(q)^{-1}J_t^T(q))^{-1}$. The dynamically consistent inverse of the task Jacobian can be written as:

$$J_t^\#(q) = M(q)^{-1}J_t^T(q)\Lambda_t(q) \quad (3)$$

Multiplying the equation of motion in the generalized coordinates by the transpose of the dynamically consistent inverse of the task Jacobian, we can get the equation of motion written in the operational space:

$$\Lambda_t(q)\ddot{x} + \mu_t(q, \dot{q}) + p_t(q) = F_t - \left(J_t^\#(q)\right)^T \tau_{ext} \quad (4)$$

where μ_t is the task space Centrifugal and Coriolis force and p_t is the gravity vector project into the task space. The command joint torque is give by $\tau_m = J_t^T(q)F_t$.

Following [11] who claim that using variable impedance control in end-effector space have intrinsic advantages, such as sample efficiency and ensures task space safety, we use

variable impedance control in task space to calculate the task control force F_t .

$$F_t = \Lambda_t(q) (K_1(x_d - x) - D_1\dot{x}) + \mu_t(q, \dot{q}) + p_t(q) + \left(J_t^\#(q)\right)^T \tau_{ext} \quad (5)$$

K_1 and D_1 are the diagonal stiffness and damping matrix generated by the RL policy. The equilibrium point or the desired pose x_d is also given by the policy.

C. Null Space Projection

The redundant degrees of freedom (DoF) allow the robot to achieve more dexterous motions. In robotics, the most commonly used method to take advantage of kinematic redundancy is the null space projection method [19] [20].

Considering a robot with a prior task and a second task, the goal of null space projection is to execute the second task as good as possible without disturbing the prior task. The Jacobian of the prior task is defined as $J_1(q)$. We can use the successive null space projection [21] to project the second task torque τ_2 into the null space of the prior task,

$$\tau_2^p = N_2^T(q)\tau_2 \quad (6)$$

where τ_2^p is the projected torque that does not disturb the prior task. The successive null space projector $N_2(q)$ is defined as:

$$N_2(q) = I - J_1^\#(q)J_1(q) \quad (7)$$

where $J_1^\#(q)$ is the generalized inverse of $J_1(q)$ which satisfies $J_1(q)J_1^\#(q) = I$. We can compute $J_1^\#(q)$ by

$$J_1^\#(q) = W^{-1}J_1^T(q) (J_1(q)W^{-1}J_1^T(q))^{-1}$$

where the common choice of W is the identity matrix I or the mass matrix $M(q)$.

III. METHODS

The framework overview of our proposed method is shown in Fig. 2. Our framework is mainly composed of two parts, the RL policy and the controller. The RL policy takes both the camera observation image and the robot state as inputs, and outputs the increment of end-effector pose, as well as the stiffness and damping matrices of the Cartesian VIC. The controller takes the output of the RL policy as the input and outputs the robot joint torques. A modified momentum observer from [22] [23] is used as the contact estimator to detect unexpected collisions on the robot arm. When no collision is detected, the controller will execute the main VIC task which maintains task space safety, while performing a posture task in the null space to regulate the joint behavior. Once collision is detected, a contact-aware controller will be activated, which will perform contact-aware null space projection to keep the external contact force small, while executing a disturbance compensation impedance task to reject the effects of external disturbances on the main task. The RL policy and the controller run at different frequencies due to the computation resource limitation. The policy can be obtained with any RL algorithm while in this work we

use proximal policy optimization (PPO) [24] with variational autoencoder (VAE) [25] pretraining. We now describe the details of our proposed method.

A. Image-Based RL Policy

In this paper, we consider a generic setting of image-based RL policy, where both the observation image and the robot state are used as the policy inputs. To improve the sampling efficiency, we firstly use a variational auto encoder (VAE) [25] to extract the latent embedding for the image in an unsupervised way. Let us denote the image as o , then the latent embedding s_{img} obtained from the VAE's encoder f_ϕ is defined as:

$$s_{img} = f_\phi(o) \quad (8)$$

The final state s is obtained from concatenating the end-effector's 6-DoF pose s_{pose} with s_{img} . Putting the state s into PPO's policy network π_ψ , we get the action a :

$$a = \pi_\psi(s) = \pi\left(\begin{bmatrix} s_{pose} \\ s_{img} \end{bmatrix}\right) \quad (9)$$

where we use variable impedance control in Cartesian space as our action space which consists of the increment of the diagonal stiffness matrix K_1 , the damping matrix D_1 and the equilibrium point x_d . Details of the training process of our RL policy will be introduced in section III.F.

B. Contact Estimator

To estimate whether a contact is happened on the robot arm, we use the momentum observer [22] [23], which is a stable, linear, decoupled, first-order estimation method of the external joint torque τ_{ext} . Define the generalized momentum p of the robot as

$$p = M(q)\dot{q} \quad (10)$$

[22] [23] suggest that the τ_{ext} can be approximated using the residual vector $\gamma \in \mathbb{R}^n$:

$$\gamma = K_I \left(p(t) - \int_0^t (\tau_m - \hat{\beta}(q, \dot{q}) + \gamma) ds - p(0) \right) \quad (11)$$

where $\hat{\beta}$ is the estimate of $\beta(q, \dot{q}) = g(q) + C(q, \dot{q}) - \dot{M}(q)\dot{q}$ and $K_I > 0$ is the diagonal gain matrix. Under ideal conditions that mass matrix $M(q)$ and $\beta(q, \dot{q})$ is well estimated, the dynamics of the residual vector and the external joint torque is given by

$$\dot{\gamma} = K_I(\tau_{ext} - \gamma) \quad (12)$$

If $K_I \rightarrow \infty$, we have $\gamma \approx \tau_{ext}$.

Using the above method we can estimate the total external joint torque, which consists of torques induced by both the unexpected contact force happened at the robot arm and the expected contact force happened at the end-effector:

$$\tau_{ext} = \tau_c + \tau_{eff} = J_c^T(q)F_c + J_t^T(q)F_{eff} \quad (13)$$

where $J_c(q)$ is the Jacobian at the contact point, F_c is the contact force at joint level, τ_c is the corresponding contact joint torque and F_{eff} is the contact force at the end-effector. Usually F_{eff} can be directly measured by the force torque

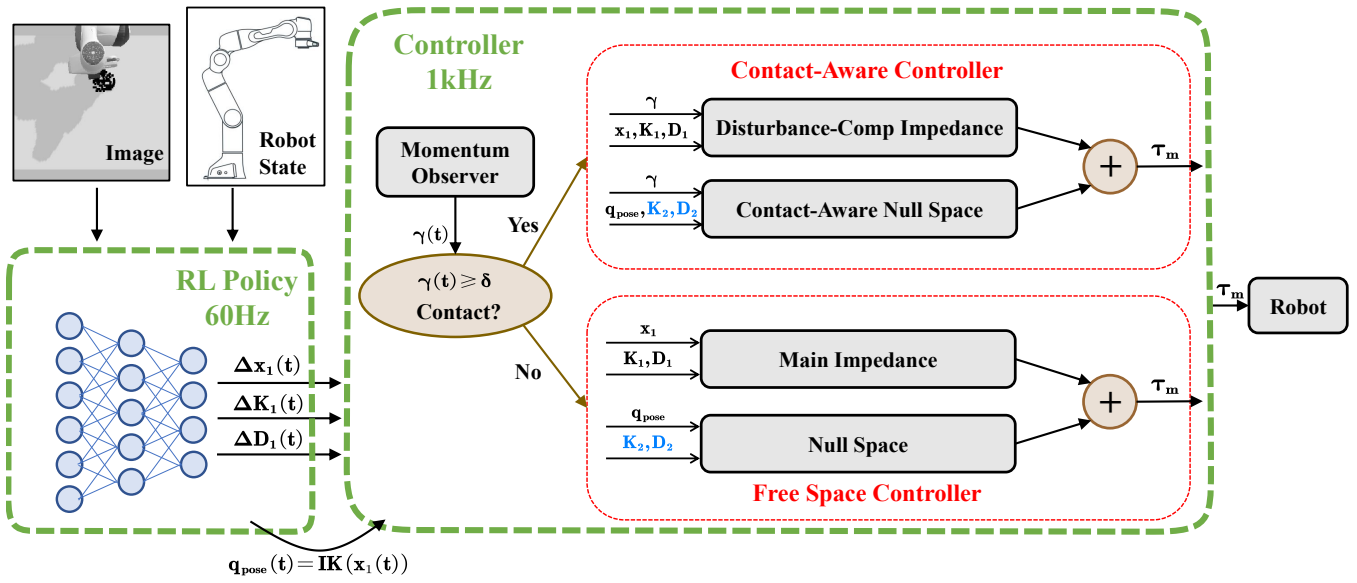


Fig. 2. Framework Overview: Our method consists of a low-rate RL policy and a high-rate controller. The RL policy takes the image and the robot eff-effector pose as input, and outputs task space information. We use inverse kinematics to define the posture task. The controller consists of a contact-aware controller and a free space controller while using momentum observer as the contact detector. The controller switches to the contact-aware controller under contact, and switches to the other on the contrary. The final command torque will be sent to the robot.

sensor. If we want to estimate the contact force applied on the arm only, we need to cancel out the end-effector contact force in the residual vector:

$$\gamma = K_I \left[p(t) - \int_0^t \left(\tau_m + J_t^T(q) F_{eff} - \hat{b}(q, \dot{q}) + \gamma \right) ds - p(0) \right] \quad (14)$$

In the free space, the residual vector should ideally be zero. But in practice, it's hard to get a precise estimation, so we assume the contact condition is triggered when the residual is greater than a predefined threshold δ . The link in contact corresponds to the coefficient of γ that is greater than δ and is the furthest away from the robot base.

C. Free Space Controller

Using variable impedance control in the end-effector space has many intrinsic benefits, but this appealing control scheme would generate unpredictable motion in the free space. As suggested in [6], we find the same problem that the arm might unexpectedly continue to accelerate. We name this problem as the joint acceleration problem when no joint task exist. The reason is that the joint control torque τ_{ext} is a projection of the task control force F_t , and there are no control targets on the robot joints. To alleviate this problem, we add an additional posture task to the robot joints.

In order to prevent the posture task from affecting the execution of the prior task, we use null space projections to execute the posture task as good as possible without disturbing or interfering the prior task. The posture task goal is to stay in a reference configuration q_{pose} :

$$\Gamma_{pose} = K_2(q_{pose} - q) - D_2\dot{q} \quad (15)$$

where K_2 and D_2 are the diagonal joint stiffness and damping matrix. The dynamic consistent null space projection

matrix is:

$$N_t(q) = I - J_t^\#(q)J_t(q) \quad (16)$$

This projector can project the posture force into the null space of the prior task. Note here the τ_{ext} term in Equation (4) is zero. The free space controller is defined as:

$$\tau_m = J_t^T(q)F_t + N_t^T(q)\Gamma_{pose} \quad (17)$$

D. Contact-Aware Controller

Using the free space controller with a well-trained RL policy will maintain the task space safety. However, the robot sometimes will encounter unexpected contact at its arm, during human interaction or working in unstructured environment. In order to keep the joint space safety, we design a contact-aware controller inspired by [26]. We use a disturbance compensation impedance to avoid the unexpected contact from disturbing the prior task, and a contact-aware null space projection to keep the contact force small between the robot arm and the environment.

1) *Disturbance Compensation Impedance*: In free space, the contact joint torque τ_c is zero. But under joint contact situation, we need to consider the effect of τ_c . The task control force can be written as:

$$F_{t-c} = \Lambda_t(q) (K_1(x_d - x) - D_1\dot{x}) + \mu_t(q, \dot{q}) + p_t(q) - \left(J_t^\#(q) \right)^T \gamma \quad (18)$$

In order to avoid disturbing the prior task defined by the Cartesian variable impedance control, we need to cancel out the joint torque induced by the expected contact force at the end-effector. Therefore, we use Equation (14) to calculate γ , which only estimate the contact joint torque τ_c .

2) *Contact-Aware Null Space Projection*: When in contact with the environment, the robot is under additional constraints which makes the robot lose degrees of freedom in the motion space. To prevent the robot from violating the constraint, we need to remove this degree of freedom from the posture task force. At the contact point, we have:

$$\tau_c = J_c^T(q)F_c \quad (19)$$

where $J_c(q)$ is the Jacobian at the contact point, F_c is the contact force and τ_c is the corresponding contact joint torque. The direction of the contact can be written as

$$u_c := \frac{F_c}{\|F_c\|} \quad J_{u_c} = u_c^T J_c(q) \quad (20)$$

The reduced contact Jacobian J_{u_c} maps the joint velocity $\dot{q}$ to the velocity along the direction of the contact u_c . If we project the posture task into the intersection of the null space of the task Jacobian and that of the reduced contact Jacobian (namely the task-consistent null space of the reduced contact Jacobian), the posture task will not violate the contact constraint as well as the prior task, which results in compliant behavior.

Practically, the task-consistent null space of the reduced contact Jacobian can be calculated as follows [26]:

$$N_{c|t} = N_t \left(I - (\gamma^T N_t)_{M-1}^\# (\gamma^T N_t) \right) \quad (21)$$

where $(\gamma^T N_t)_{M-1}^\#$ is the generalized inverse of $\gamma^T N_t$ weighted by the inverse of the mass matrix.

Now the control law becomes:

$$\tau_m = J_t^T(q)F_{t-c} + N_{c|t}^T \Gamma_{pose} \quad (22)$$

The total control law of our controller part is:

- No contact (Free Space Controller):

$$\tau_m = J_t^T(q)F_t + N_t^T(q)\Gamma_{pose}$$

- With contact (Contact-Aware Controller):

$$\tau_m = J_t^T(q)F_{t-c} + N_{c|t}^T \Gamma_{pose}$$

E. Coordinates between RL Policy and controller

As shown in Fig.2, our framework is hierarchical, where the upper level is the RL policy and the lower level is the controller. The RL policy generates the increment of the equilibrium point, the task space diagonal stiffness and damping matrix. After obtaining x_d from the policy, we can perform inverse kinematics (IK) to define the posture task:

$$q_{pose} = \text{IK}(x_d) \quad (23)$$

The high-level RL policy runs at a lower rate (e.g. 60Hz), while the low-level torque controller runs at a higher rate (e.g. 1000Hz). We observe that the controller has a jittering phenomenon under some disturbance contact caused by: (1) the oscillation of the residual vector and (2) the switching between the two sub-controllers. We add a low-pass filter and set the gain K_I of the residual vector small to mitigate the first problem. We alleviate the second problem by enforcing that the controller will not be switched until the execution time of the contact-aware controller is greater than the threshold t_c .

F. Training of RL Policy

We train our policy in the simulation with no unexpected contact on the robot arm. We first pretrain a VAE model from randomly sampled possible observations with the loss function:

$$\mathcal{L}_{\text{VAE}}(\theta, \phi; o, s_{img}, \beta) = \mathbb{E}_{q_\phi(s_{img}|o)} [\log p_\theta(o|s_{img})] - \beta D_{KL}(q_\phi(s_{img}|o) \| p(s_{img})) \quad (24)$$

where $D_{KL}(\cdot \| \cdot)$ is the KL Divergence between two distributions, θ is the parameters of the decoder network, the the encoder $f_\phi(\cdot)$ is the mean of the variational distribution $q_\phi(s_{img}|o)$ and β is a hyperparameter.

The RL policy is trained with the free space controller as described in section III.C using PPO [24]. The training objectives is to maximize:

$$\mathcal{L}(\psi) = \mathbb{E}_t [\min(r_t(\psi)A_t, \text{clip}(r_t(\psi), 1 - \epsilon, 1 + \epsilon)A_t)] \quad (25)$$

where A_t is the advantage function, ϵ is a hyperparameter, ψ is the parameter of the policy network. $r_t(\psi)$ is defined as:

$$r_t(\psi) = \frac{\pi_\psi(a_t|s_t)}{\pi_{\psi_{\text{old}}}(a_t|s_t)} \quad (26)$$

where ψ_{old} is the vector of policy parameters before the update.

IV. EXPERIMENTS

Among multiple contact-rich tasks, we choose surface wiping as our experimental showcase. Our agent's goal is to wipe the ink drawn on the whiteboard. While wiping the surface, we intentionally cause unexpected contacts on the robot arm, and compare both task space safety and joint space safety with the baseline method.

A. Implementation Details

We implement our method on a real Franka Panda 7 DoF robot arm with a two-finger gripper. We use AMD 5800H and NVIDIA RTX3070 as run-time CPU and GPU with an Intel RealSense D455 camera. We train our agent only in simulation, then deploy the learned policy in real environment with zero-shot. We choose pybullet [27] as our simulator.

For each training episode, ink is randomly generated on the simulated whiteboard. The observation space incorporates both end-effector's 6-DoF pose and a $128 \times 128 \times 1$ image. In order to close the sim2real gap of the image, we do segmentation both on real and simulation, where pixels inside the ink are set to zero while others are set to 255. For image feature extraction, we pretrain a VAE model, which encodes an image to a 128-dimension vector for the downstream RL tasks. Our agent gets a reward proportional to the inks wiped off, and an additional reward when all inks are wiped. To encourage exploration at the beginning, a small reward is given if the agent keeps contacting the table and the contact force is less than $40N$. Moreover, to guide the end-effector toward the ink, we provide a small reward increasing as the distance between the end-effector's position and the ink

decreases. When the robot arm collides with the table, we give a large penalty and the episode terminates.

When transferred to the real world, the RL policy runs at 60Hz and the controller runs at 1000Hz. We use [28] as our inverse kinematics solver. The momentum observer threshold δ is $1.5Nm$ and diagonal gain matrix K_I is set as $1.5I$ where I is the identity matrix. For posture task, we set the stiffness K_2 for each joint as $[40, 40, 40, 40, 30, 20, 10]$ and set the damping D_2 for each joint as $[12, 12, 12, 12, 10, 8, 6]$. The controller switch threshold t_c is set as $50ms$.

Our baseline follows the VICES framework [11], except that a posture task is added for avoiding the accelerate problem, and all the parameters are set the same with our method for a fair comparison. In general, VICES considers task space safety but does not consider joint space safety.

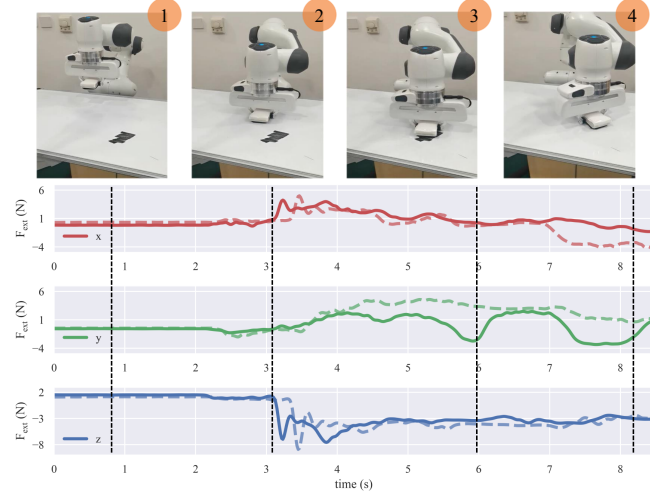


Fig. 3. Task Space Safety: The bottom three figures plot the 3-axis end-effector force when the robot wipes the whiteboard. The safe task contact is defined as the contact force under $40N$. The maximum force of our methods is about $5N$ which is far less than the safe contact threshold, while performing the wiping task successfully. The black dashed lines mark the time when the situation shown in the top images occurred and other colored dashed line plots the result of the baseline.

B. Task Space Safety

We firstly show the task space safety of our proposed method, which is defined by how large the end-effector contact force is when performing the contact-rich manipulation task. We manually draw a set of inks on the table and let the robot wipe the inks actuated by the trained RL policy and the controller. The result is shown in Fig. 3. The maximum force of the Franka Panda robot arm can apply is $40N$. In our setting, same as VICES, the unsafe situation is defined as if the task contact force at the end-effector is greater than $40N$. We can see that the robot can finish the wiping task successfully while keeping a safe contact force. Note both our proposed method and the baseline maintains compliant contact since the baseline also considers task space safety. Our method shows similar performance with the baseline since their only difference is the use of posture task when focusing on task space safety. This result shows that the posture task does not affect the main task.

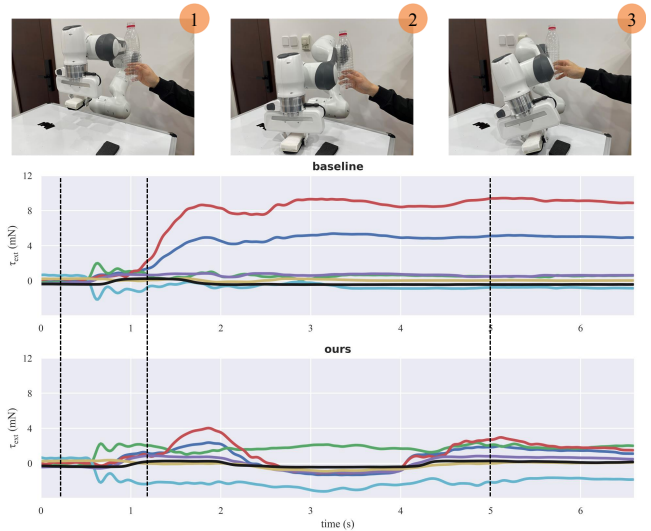


Fig. 4. Joint Space Safety: The bottom two figures plot the estimated external joint contact force induced by unexpected contact at the robot arm.

C. Joint Space Safety

To illustrate that our method can maintain the joint space safety with an RL policy, we hold a plastic water bottle at the position where the robot will pass through as the robot see the unseen black styrofoam, as shown in Fig. 1. In our experiment, the robot will successfully wipe the ink if no black styrofoam was put in place. However, if the black styrofoam was placed, the robot will go right and hit the plastic water bottle we hold. The black styrofoam thus acts as a disturbance for the RL agent.

The bottom figure in Fig. 1 intuitively shows the contact force. The green border is ours and the red one is the baseline. Our method does not deform the plastic water bottle at all while the baseline deforms it significantly. We also use a thrust meter to measure the actual contact force quantitatively at the contact point, holding the thrust meter at the same position as the water bottle. The contact force of our method shown in the thrust meter is 3 times less than the baseline. Fig. 4 shows that the estimated external joint contact force in our methods is less than the baseline: the baseline reaches $10Nm$ but our method is less than $5Nm$.

We can see that our method generates far less contact force and shows a much more compliant behavior. This compliant behavior can provide better safety guarantees, which is crucial for RL agents in the real world, especially when the agent visits an unexperienced state, or is in the exploration phase where random actions may lead to dangers. Our compliant controller protects both the robot and the surrounding environment (including humans).

D. Task Disturbance Rejection

When undergoing unexpected contacts, it is desirable if the robot can perform a compliant behavior while rejecting disturbances and keep the prior task as precise as possible. We show that our method has better accuracy on the prior task while maintaining compliance. In our experiments, we randomly disturb the robot by pushing it when it is working, shown as the arrows in Fig. 5. We can see that the baseline

method is much more easier to get affected, reaching a 4cm maximum error in the y direction. However, the error of our method is less than 1cm in all directions. This shows that our method can reject the impact disturbances on the prior task due to unexpected contacts.

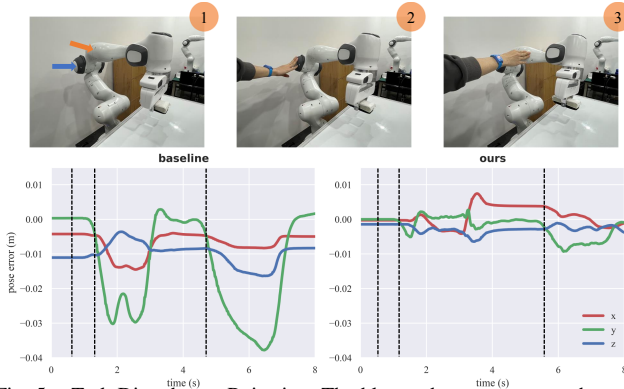


Fig. 5. Task Disturbance Rejection: The blue and orange arrows show the disturbance force applied by human. The blue one applied first while the orange one applied next. The bottom right figure shows the end-effector position error of our method under the unexpected contact. The bottom left one shows the result of the baseline.

V. CONCLUSION

This work focuses on using reinforcement learning in contact-rich manipulation tasks while keeping both task space and joint space contact safety, which is important due to the potential risks in both the exploration and deployment procedures of RL. We proposed a hierarchical framework with an RL policy to generate the cartesian VIC target, and a controller to perform this task while keeping compliant for both the end-effector task contact and the robot arm unexpected contact. We trained our policy in the simulation and deployed it on a real Franka Panda 7 DoF robot. The whiteboard wiping experiments show that our method is able to perform compliant behaviors under unexpected contacts caused by unexpected RL policy outputs. Results also suggest that the proposed framework maintains safe contact force when performing the wiping task, while rejecting the disturbances caused by unexpected joint contacts. Although focusing on the wiping task using a specific RL algorithm in this work, our framework is generic and can be applied to other tasks with different RL algorithms.

REFERENCES

- [1] D. Kalashnikov, A. Irpan, P. Pastor, J. Ibarz, A. Herzog, E. Jang, D. Quillen, E. Holly, M. Kalakrishnan, V. Vanhoucke, *et al.*, “Scalable deep reinforcement learning for vision-based robotic manipulation,” in *Conference on Robot Learning*, pp. 651–673, PMLR, 2018.
- [2] S. Levine, P. Pastor, A. Krizhevsky, J. Ibarz, and D. Quillen, “Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection,” *The International journal of robotics research*, vol. 37, no. 4-5, pp. 421–436, 2018.
- [3] C. C. Beltran-Hernandez, D. Petit, I. G. Ramirez-Alpizar, T. Nishi, S. Kikuchi, T. Matsubara, and K. Harada, “Learning force control for contact-rich manipulation tasks with rigid position-controlled robots,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, 2020.
- [4] O. Spector and M. Zacksenhouse, “Deep reinforcement learning for contact-rich skills using compliant movement primitives,” *arXiv preprint arXiv:2008.13223*, 2020.
- [5] G. Thomas, M. Chien, A. Tamar, J. A. Ojea, and P. Abbeel, “Learning robotic assembly from cad,” in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3524–3531, IEEE, 2018.
- [6] J. Luo, E. Solowjow, C. Wen, J. A. Ojea, A. M. Agogino, A. Tamar, and P. Abbeel, “Reinforcement learning on variable impedance controller for high-precision robotic assembly,” in *2019 International Conference on Robotics and Automation (ICRA)*, pp. 3080–3087, IEEE, 2019.
- [7] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. 2018.
- [8] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, “Exploration by random network distillation,” *arXiv preprint arXiv:1810.12894*, 2018.
- [9] N. Hogan, “Impedance control: An approach to manipulation: Part i—theory,” 1985.
- [10] F. J. Abu-Dakka and M. Saveriano, “Variable impedance control and learning—a review,” *Frontiers in Robotics and AI*, p. 177, 2020.
- [11] R. Martín-Martín, M. A. Lee, R. Gardner, S. Savarese, J. Bohg, and A. Garg, “Variable impedance control in end-effector space: An action space for reinforcement learning in contact-rich tasks,” in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1010–1017, IEEE, 2019.
- [12] L. Roveda, J. Maskani, P. Franceschi, A. Abdi, F. Braghin, L. Molinari Tosatti, and N. Pedrocchi, “Model-based reinforcement learning variable impedance control for human-robot collaboration,” *Journal of Intelligent & Robotic Systems*, vol. 100, no. 2, pp. 417–433, 2020.
- [13] Q. Yang, J. A. Stork, and T. Stoyanov, “Null space based efficient reinforcement learning with hierarchical safety constraints,” in *2021 European Conference on Mobile Robots (ECMR)*, pp. 1–6, IEEE, 2021.
- [14] S. Shaw, B. Abbatematteo, and G. Konidaris, “Rmps for safe impedance control in contact-rich manipulation,” *arXiv preprint arXiv:2109.12103*, 2021.
- [15] X. Zhao, T. Fan, Y. Li, Y. Zheng, and J. Pan, “An efficient and responsive robot motion controller for safe human-robot collaboration,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, 2021.
- [16] M. El-Shamouty, X. Wu, S. Yang, M. Albus, and M. F. Huber, “Towards safe human-robot collaboration using deep reinforcement learning,” in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4899–4905, IEEE, 2020.
- [17] C. Eppner, R. Deimel, J. Alvarez-Ruiz, M. Maertens, and O. Brock, “Exploitation of environmental constraints in human and robotic grasping,” *The International Journal of Robotics Research*, vol. 34, no. 7, pp. 1021–1038, 2015.
- [18] O. Khatib, “A unified approach for motion and force control of robot manipulators: The operational space formulation,” *IEEE Journal on Robotics and Automation*, vol. 3, no. 1, pp. 43–53, 1987.
- [19] Y. Nakamura, H. Hanafusa, and T. Yoshikawa, “Task-priority based redundancy control of robot manipulators,” *The International Journal of Robotics Research*, vol. 6, no. 2, pp. 3–15, 1987.
- [20] A. Dietrich, C. Ott, and A. Albu-Schäffer, “An overview of null space projections for redundant, torque-controlled robots,” *The International Journal of Robotics Research*, vol. 34, no. 11, pp. 1385–1400, 2015.
- [21] G. Antonelli, “Stability analysis for prioritized closed-loop inverse kinematic algorithms for redundant robotic systems,” *IEEE Transactions on Robotics*, vol. 25, no. 5, pp. 985–994, 2009.
- [22] A. De Luca and R. Mattone, “Sensorless robot collision detection and hybrid force/motion control,” in *Proceedings of the 2005 IEEE international conference on robotics and automation*, pp. 999–1004, IEEE, 2005.
- [23] S. Haddadin, A. De Luca, and A. Albu-Schäffer, “Robot collisions: A survey on detection, isolation, and identification,” *IEEE Transactions on Robotics*, vol. 33, no. 6, pp. 1292–1312, 2017.
- [24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [25] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner, “beta-vae: Learning basic visual concepts with a constrained variational framework,” 2016.
- [26] M. Jorda, E. G. Herrero, and O. Khatib, “Contact-driven posture behavior for safe and interactive robot operation,” in *2019 International Conference on Robotics and Automation (ICRA)*, pp. 9243–9249, IEEE, 2019.
- [27] “Pybullet, a python module for physics simulation for games, robotics and machine learning,” <http://pybullet.org>, 2016–2021.
- [28] P. Beeson and B. Ames, “Trac-ik: An open-source library for improved solving of generic inverse kinematics,” in *2015 IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids)*, pp. 928–935, IEEE, 2015.